

Qur'an Learning Platform

Technical Architecture & Delivery Plan

A community platform connecting learners with verified teachers for live, in-platform Qur'an instruction — one-to-one and group video sessions, session recording, and an on-demand lesson library.

Prepared for internal review and technical consulting session
August 28, 2026

Executive Summary

This platform connects Muslim learners worldwide with verified teachers for Qur'an study — one-to-one and group sessions, live video calls hosted natively inside the platform, session recordings, and a growing library of on-demand lessons. Two constraints shape every decision below: real budget is available, but leadership has asked explicitly to minimize infrastructure cost, and no part of the calling experience may route through a third-party product such as Zoom or Google Meet.

This document lays out the resulting architecture, the engineering work it implies, and a phased sprint plan to deliver it.

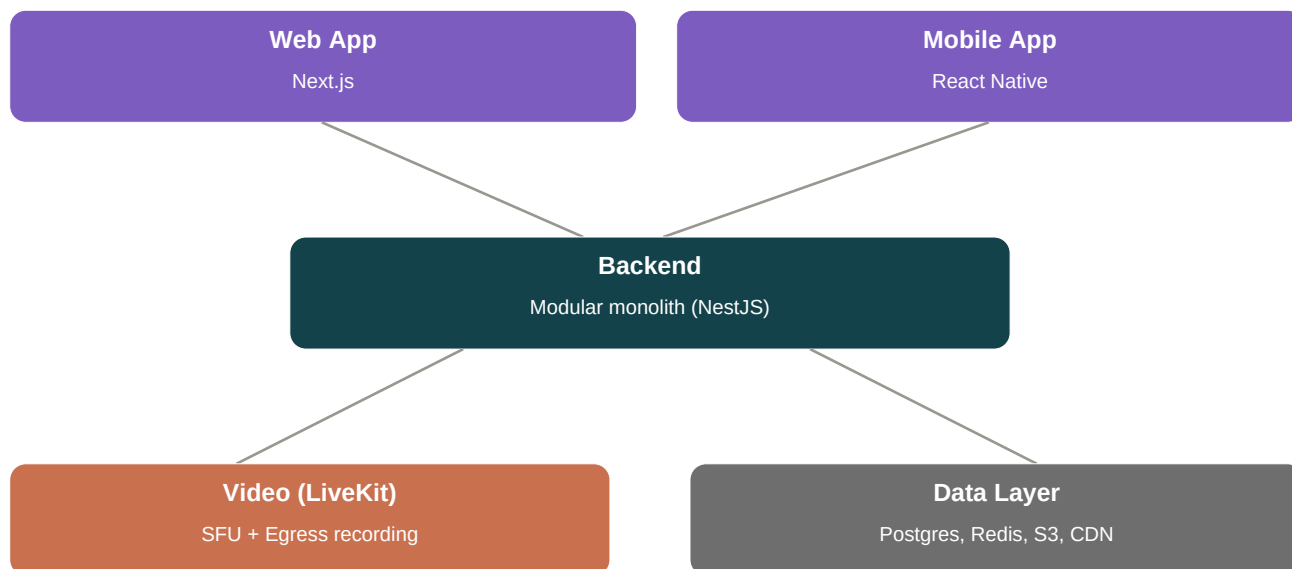
Guiding Principles

- Modular monolith over microservices until real scale demands otherwise — fewer moving parts to build, run, and pay for at this stage.
- Buy the hard 20% of real-time video (LiveKit) rather than rebuild it, with a self-hosted exit ramp that requires no application rewrite.
- Managed services first, self-hosted infrastructure once usage volume actually justifies the switch — not on principle.
- One unified video pipeline serves both call recordings and uploaded lesson content; a recording is just another video.
- Cloud provider choice is driven by data-residency requirements, not default habit.

1. System Architecture

1.1 Architecture at a Glance

Client apps talk to a modular monolith backend. The backend issues LiveKit rooms and access tokens for real-time calls, and manages a data layer of PostgreSQL, Redis, and object storage. Recordings and uploaded lesson videos both feed one shared transcoding and delivery pipeline.



1.2 Frontend

- Web: Next.js (React) — server rendering benefits public/discoverable pages (teacher profiles, published recordings).
- Mobile: React Native — one codebase for iOS and Android rather than two native builds.
- Arabic/RTL and multi-language support designed in from the start, not retrofitted.
- LiveKit's web and React Native SDKs power the in-app call UI.

1.3 Backend

Node.js / TypeScript with NestJS — its module system maps directly onto domain boundaries, keeping the monolith organized as it grows. Core modules: Identity and Profiles, Community and Matching, Sessions and Scheduling, Video Orchestration, Recording and VOD, Notifications, Admin and Moderation. Authentication supports both email and phone/OTP — phone login is often the more natural default for this audience.

1.4 Video Calls and Real-Time Communication

- Real-time media runs on LiveKit, an open-source SFU that forwards streams rather than transcoding them.
- A scheduled meeting maps directly to one LiveKit room; one-to-one and group calls use the identical mechanism — only the number of issued tokens differs.
- Access is enforced with short-lived, server-minted tokens scoped to a room and to publish/subscribe permissions — the same mechanism that distinguishes open from closed sessions.
- Silent 'watch only' participants join the same room with publish permission withheld, supporting a discoverable open session with a live audience.

- Deployment path: start on LiveKit Cloud (managed, pay per participant-minute, zero ops burden), migrate later to self-hosted LiveKit on owned infrastructure without rewriting application code, once usage volume changes the cost calculus.

1.5 Call Recording

LiveKit's Egress service records a room server-side, running as a fully separate process from the live call path so recording load can never degrade call quality. Room Composite mode renders the session through a template matching the platform's own UI and writes the output directly to object storage. A recording inherits the access level of its source session, so a closed session's recording stays private.

1.6 Video Upload and VOD Pipeline

Teachers can upload pre-recorded lessons independent of any live session. Upload goes directly from the browser to object storage via a pre-signed URL, never proxied through the application backend. Both call recordings and direct uploads feed one shared transcoding pipeline that produces adaptive-bitrate output for smooth playback across device and connection quality, then serves it through a CDN — both live in the same video library and metadata table.

1.7 Data Layer

- PostgreSQL — primary store for users, teacher credentials, sessions, matching data, and video metadata; this data is inherently relational.
- Redis — caching, live presence for matching/discovery, pub/sub for real-time UI updates.
- Object storage (S3-compatible) — recordings, uploads, avatars.
- Search — PostgreSQL full-text search initially; dedicated search infrastructure only once usage demonstrates the need.

1.8 Cloud Infrastructure and Data Residency

If in-Kingdom data residency is a hard requirement, the live hyperscaler options today are Oracle Cloud (Jeddah, Riyadh) and Google Cloud (Dammam); AWS's Saudi region is announced but not yet live, and Microsoft's is confirmed for customer workloads starting Q4 2026. If residency is a preference rather than a requirement, AWS and Azure's established UAE regions are a viable near-term alternative. For compute, a managed container platform is the right starting point for the core application — not self-managed Kubernetes, which is introduced later specifically for self-hosted LiveKit, once that migration happens.

1.9 Domain-Specific Considerations

- Qur'an text, translations, and recitation audio: integrate an established free API (Quran.com, Al Quran Cloud, Tanzil) rather than building this corpus in-house.
- Teacher verification: the data model includes credential fields (such as an ijazah chain) from day one — credibility is core to trust on this platform.
- Optional gender-preference filter for matching/booking — common in this space, cheap to build into the data model now, expensive to retrofit later.
- Moderation tooling (report, mute, remove-participant, host controls) is required from day one for open/public sessions.

1.10 Scaling Path (Post-MVP)

- Large public broadcasts move from direct WebRTC subscription to Egress → HLS → CDN, decoupling audience size from SFU node capacity entirely.
- Multi-region node pools once the audience is geographically distributed enough for latency to matter.

- Self-hosted LiveKit migration once usage volume crosses the point where infrastructure cost undercuts managed per-minute pricing.

2. Work Breakdown Structure

Engineering scope grouped into nine epics. Epics 1–5 build the core learning and calling experience; 6–7 build community and trust; 8 is the infrastructure backbone running throughout; 9 is explicitly post-MVP.

Epic	Key Tasks
Foundation and Identity	• Auth (email + phone/OTP), session management • User and teacher profile data models • Teacher verification/credential workflow • i18n scaffolding (Arabic RTL + English) • Base design system / component library
Scheduling and Meetings	• Meetings and meeting_participants data model • Create / update / cancel meeting API • Booking UI + calendar view • Reminder notifications, timezone handling
Video Calls (Core)	• LiveKit Cloud integration, server-side token service • Room lifecycle webhook handlers • Call UI: join flow, video grid, mic/camera controls • Open vs. closed session access control • Silent/observer join mode
Call Recording	• Egress integration (Room Composite) • Recording metadata linked to source session • Access control inheritance (closed → private) • Storage lifecycle and retention policy
Video Upload and VOD Library	• Pre-signed upload endpoint, direct-to-storage flow • Transcode pipeline integration • Unified videos table (recordings + uploads) • Library browsing UI + adaptive-bitrate player
Community and Matching	• Teacher discovery / directory with filters • Study circle (group) creation and membership • Interest/preference-based matching (search first)
Trust and Moderation	• Report / mute / remove-participant tooling • Teacher approval workflow (admin review queue) • Basic admin dashboard (users, sessions, reports)
Cloud Infrastructure and DevOps	• Cloud environment provisioning (region — see 1.8) • CI/CD pipelines (backend, frontend, mobile) • Monitoring, logging, alerting • Staging / production environment separation
Scale-Out (Post-MVP)	• Egress → HLS → CDN path for large broadcasts • Self-hosted LiveKit migration (Compose → Kubernetes) • Multi-region node pools

3. Sprint Plan

Planning assumption: a team of about five to six people (two backend, one to two frontend/mobile, one full-stack/DevOps, one QA/PM-adjacent), working two-week sprints. Pacing scales roughly inversely with team size — adjust accordingly.

Sprint	Phase / Goal	Key Deliverables
1–2 Wks 1–4	Foundation	Cloud environment + CI/CD running; backend and frontend skeletons deployed; auth (email + OTP) working end-to-end; teacher/user profile model; i18n/RTL scaffolding in place.
3–4 Wks 5–8	Core Calling MVP	Meetings data model + booking UI; LiveKit Cloud integrated; token service + join flow; working call UI (audio/video, controls); open vs. closed access control.
5–6 Wks 9–12	Recording and Video Library	Egress recording live; upload flow + transcode pipeline; unified video library with adaptive playback; recording access control.
7–8 Wks 13–16	Community and Trust	Teacher directory + basic matching; study circles; moderation tooling; teacher approval workflow; admin dashboard v1.
9 Wks 17–18	Hardening and Launch	Security review; load testing of the call path; bug bash; monitoring/alerting in production; soft launch.
Post-launch	Scale-Out (not committed sprints)	Broadcast/CDN path for large audiences; self-hosted LiveKit migration; multi-region expansion — triggered by real usage, not the calendar.

4. Open Questions and Decisions Needed

- Expected scale at launch and at 12 months (concurrent sessions, total users) — determines whether and when self-hosting the video layer pays off.
- Is in-Kingdom data residency a hard compliance requirement or a preference?
- Monetization model — free platform, teacher-paid, subscription, or donation-based? Not currently scoped; affects payments work.
- Is a gender-preference filter for matching/booking an explicit requirement?
- Confirm team size and composition — the sprint plan assumes ~5–6 people.

5. Glossary

SFU (Selective Forwarding Unit)	The server model behind the calls — routes each participant's stream to the others without decoding or re-encoding it, which is what makes group calls efficient.
WebRTC	The browser/mobile standard for real-time audio and video, used for the live call path.
TURN / STUN	Helper servers that let devices behind restrictive networks reach the call server; STUN handles the easy majority, TURN relays media for the rest.
Egress	LiveKit's recording/export service — captures a live session and writes it out as a file or stream, running separately from the live call path.
HLS (HTTP Live Streaming)	The format used to deliver recorded/uploaded video smoothly across devices and connection speeds, and the mechanism used for large passive audiences.
Modular Monolith	One deployable backend, internally organized into clean, independent domain modules — faster to build and run than microservices at this stage, and easy to split apart later if needed.
CDN (Content Delivery Network)	A globally distributed cache that serves video and static files from a location close to each viewer.